Solution: Problem Challenge 1: Quadruple Sum to Target

Problem Statement

Given an array of unsorted numbers and a target number, find all **unique quadruplets** in it, whose **sum is equal to the target number**.

Example 1:

```
Input: [4, 1, 2, -1, 1, -3], target=1
Output: [-3, -1, 1, 4], [-3, 1, 1, 2]
Explanation: Both the quadruplets add up to the target.
```

Example 2:

```
Input: [2, 0, -1, 1, -2, 2], target=2
Output: [-2, 0, 2, 2], [-1, 0, 1, 2]
Explanation: Both the quadruplets add up to the target.
```

Solution

This problem follows the **Two Pointers** pattern and shares similarities with "Triplet Sum to Zero".

We can follow a similar approach to iterate through the array, taking one number at a time. At every step during the iteration, we will search for the quadruplets similar to Triplet Sum to Zero whose sum is equal to the given target.

Here's a detailed walkthrough of the algorithm:

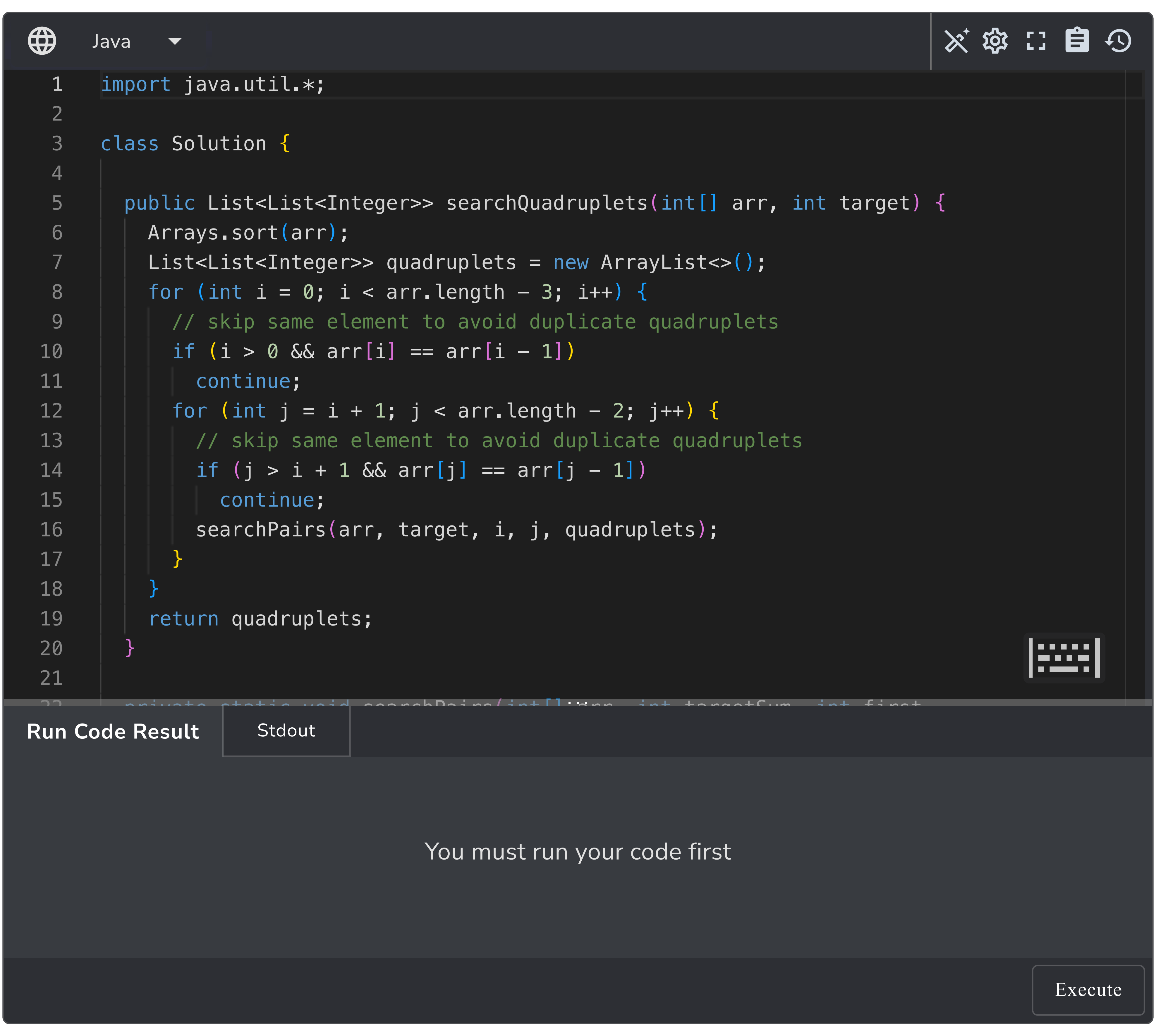
1. In `searchQuadruplets`, the array is first sorted in ascending order. Sorting is important as it allows us to navigate our pointers based on the sum we're calculating and ensures that the generated quadruplets are in a predictable order.
2. A `List` named `quadruplets` is created to store all the quadruplets found.
3. The algorithm then loops over the array, stopping when there are less than four elements remaining (since we need groups of four).
4. If the current element is the same as the previous one (and it's not the first), we skip this iteration to avoid duplicate quadruplets.
5. A nested loop is initiated from the next index of the outer loop. This loop also ensures that the current element isn't the same as the previous one to avoid duplicates.
6. The `searchPairs` function is called with the array, target value, indices of the first two elements, and the `quadruplets` list. This function will find pairs in the array (from `second+1` index to the end) whose sum with `arr[first]` and `arr[second]` equals the target. Any valid quadruplets found are added to the `quadruplets` list.
7. In `searchPairs`, two pointers `left` and `right` are initialized: `left` to `second+1`, and `right` to the last element of the array. It then enters a while loop that continues until `left` is less than `right`.
8. Inside this loop, the sum of the elements at the current four indices (`first`, `second`, `left`, `right`) is calculated. If this sum equals `targetSum`, a valid quadruplet is found.
9. This quadruplet is added to the `quadruplets` list, and both `left` and `right` pointers are moved inward. If the next elements are the same as the current elements of `left` and `right` respectively, they are skipped to avoid duplicates.
10. If the calculated sum is less than `targetSum`, `left` is incremented to increase the sum (as the array is sorted), and if the sum is greater than `targetSum`, `right` is decremented to decrease the sum.
11. This process repeats until `left` and `right` cross, by which point all possible pairs for the given `first` and `second` indices have been examined.
12. Once `searchPairs` has processed all possible pairs for the given `first` and `second` indices, it returns, and the nested loop in `searchQuadruplets` continues until all possible starting points for quadruplets have been tried.
13. Once all possible quadruplets have been considered, `searchQuadruplets` returns the `quadruplets` list.

The `main` function in the code demonstrates usage of the `searchQuadruplets` function with two test cases. It runs `searchQuadruplets` with specified arrays and target sums, printing the results to the console.

Let's walkthrough the example 1 through diagram below.

Code

Here is what our algorithm will look like:



```java
import java.util.*;

class Solution {

  public List<List<Integer>> searchQuadruplets(int[] arr, int target) {
    Arrays.sort(arr);
    List<List<Integer>> quadruplets = new ArrayList<>();
    for (int i = 0; i < arr.length - 3; i++) {
      // skip same element to avoid duplicate quadruplets
      if (i > 0 && arr[i] == arr[i - 1])
        continue;
      for (int j = i + 1; j < arr.length - 2; j++) {
        // skip same element to avoid duplicate quadruplets
        if (j > i + 1 && arr[j] == arr[j - 1])
          continue;
        searchPairs(arr, target, i, j, quadruplets);
      }
    }
    return quadruplets;
  }

```

Time Complexity

Sorting the array will take $O(N * logN)$. Overall searchQuadruplets() will take $O(N * logN + N^3)$, which is asymptotically equivalent to $O(N^3)$.

Space Complexity

The space complexity of the above algorithm will be $O(N)$ which is required for sorting.

Please wait! Loading...

← Previous
Problem Challenge 1: Quadruple Sum To Target (Medium)

Next →
Problem Challenge 2: Comparing Strings Containing Backspaces (Medium)

☐ Mark as Completed